



42 ABU DHABI · CAPSTONE PROJECT · STAFF EVALUATION

Capstone Project: Ft_Transcendence

Salim Hamzaoui · shamzaou

Alisher Abdullaev · alabdull

Nasser Alzaabi · naalzaab

Nour Murat · nurmurat

FAST_PONG — a web platform for 3D Pong and Tic-Tac-Toe, tournaments with matchmaking, player statistics, 42 login, 2FA and GDPR tools

7 Major + 6 Minor modules = 10 major-equivalents · Evaluation date: ____ / ____ / 2026

CONTENTS

01

Introduction

Nour

02

Software Development Life
Cycle

Alisher

03

Selected Modules & Design

Salim

04

Authentication & Security

Nasser

05

Games & Graphics

Salim

06

Tournaments

Nour

07

Profiles, Statistics & Friends

Alisher

08

GDPR & Accessibility

Nasser

09

Team & Conclusion

Alisher

01

Introduction

Presenter: **Nour Murat**



Project Overview

Ft_transcendence is a web-based multiplayer gaming platform developed as the capstone project of the 42 Abu Dhabi curriculum. Its core is a modern 3D implementation of the classic Pong, joined by a second game — Tic-Tac-Toe — and a complete user experience: secure accounts, 42 Intra login, email two-factor authentication, player profiles with statistics and match history, a friends list, a tournament system and GDPR tools.

The application is a Single Page Application: Django renders the first page on the server, then JavaScript swaps views without full reloads. The backend is Django (Python) with a PostgreSQL database; Gunicorn serves the site over HTTPS, and the whole stack runs in Docker Compose. The 3D game is rendered with Three.js (WebGL) and includes a computer-controlled opponent.

Backend

Django 4.2, Django REST Framework, SimpleJWT, Gunicorn (HTTPS on port 443)

Frontend

Vanilla JavaScript SPA, Bootstrap toolkit, Three.js for the 3D Pong scene

Data & Ops

PostgreSQL 13, Docker Compose, Git/GitHub feature-branch workflow



Project Objectives

The goal was to design, develop and deploy a fully functional and secure web application centred on a Pong game. The objectives set at the outset were:

Functional gaming platform

A fully operational website with 3D Pong (two players or versus AI) and a second game, Tic-Tac-Toe, played locally on one device; every game is recorded in the player's history.

Secure authentication

Email/password registration, 42 Intra OAuth for students, optional email-based 2FA and JWT tokens.

Persistent user profiles

Display name, avatar, friends, win/loss statistics, best score and a complete match history for both games.

Tournament mode

Create a tournament of 3–8 nicknames, play a round-robin of Pong matches and determine a winner.

Application security & GDPR

Protection against SQL injection, XSS and CSRF; data export, anonymization and account deletion.

Collaborative development

An Agile, feature-branch workflow with code reviews, Docker and a maintainable codebase.

02

Software Development Life Cycle

Presenter: **Alisher Abdullaev**



Chosen SDLC Model: Agile (iterative & incremental)

The project was built in short cycles, each delivering one working feature that was merged and tested before the next one started. This suited a four-person learning project with evolving requirements.

1 Iterative development

Work was broken into features — authentication, Pong core, tournaments, profiles — each built in its own short cycle.

2 Incremental delivery

A runnable, testable version existed after every merge; functionality grew with each iteration.

3 Flexibility

Requirements and the technical approach were refined as our understanding of the project grew.

4 Collaboration & feedback

Daily check-ins, pairing on complex features and peer review of every merge.

Evidence in the repository: 86 commits between 10 Feb and 2 Apr 2025, 15 pull requests merged from feature branches (db-connect, game-setup, tournaments, profile-page, secure-cookies, OAuth, user-settings, delete-account ...), plus the 2026 pre-evaluation sweeps.



Phases and Iterations

We did not use formal time-boxed sprints, but every major feature went through the same cycle of phases:

1 Planning

Discuss the next feature (e.g. 2FA, tournament logic), clarify objectives and acceptance criteria.

2 Design

Sketch data models, API endpoints and simple wireframes before coding.

3 Implementation

Develop the feature in a dedicated Git branch to avoid conflicts with the main codebase.

4 Developer testing

Unit tests for backend logic and functional checks by the developer.

5 Integration & review

Peer code review, then merge of the feature branch into master.

6 System testing

Test the feature inside the whole application to catch regressions.



Challenges and Lessons Learned

Not everything went to plan — two setbacks shaped how we finished the project.

We aimed for microservices

We first planned the DevOps **microservices** module — splitting the backend into separate services. In practice the containers could not communicate and coordinate reliably, and wiring them together ate into our time.

So we chose a modular monolith

We stepped back to one Django service with three clean apps (userapp, gameapp, tournaments). It gave us the same separation of concerns, but it actually ran — a working monolith beats a half-built microservices setup.

We lost work on GitHub

Early on we did not use Git well together: we edited the same files without branches, so merges overwrote each other and some progress was lost.

Lesson learned

One branch per feature, always pull before you push, and merge only through a reviewed pull request. After we adopted that discipline we never lost work again — and it is exactly why we can trust our history today.

Project Timeline (Gantt Chart)

The Gantt chart planned and tracked the project over time: task durations, dependencies and milestones.

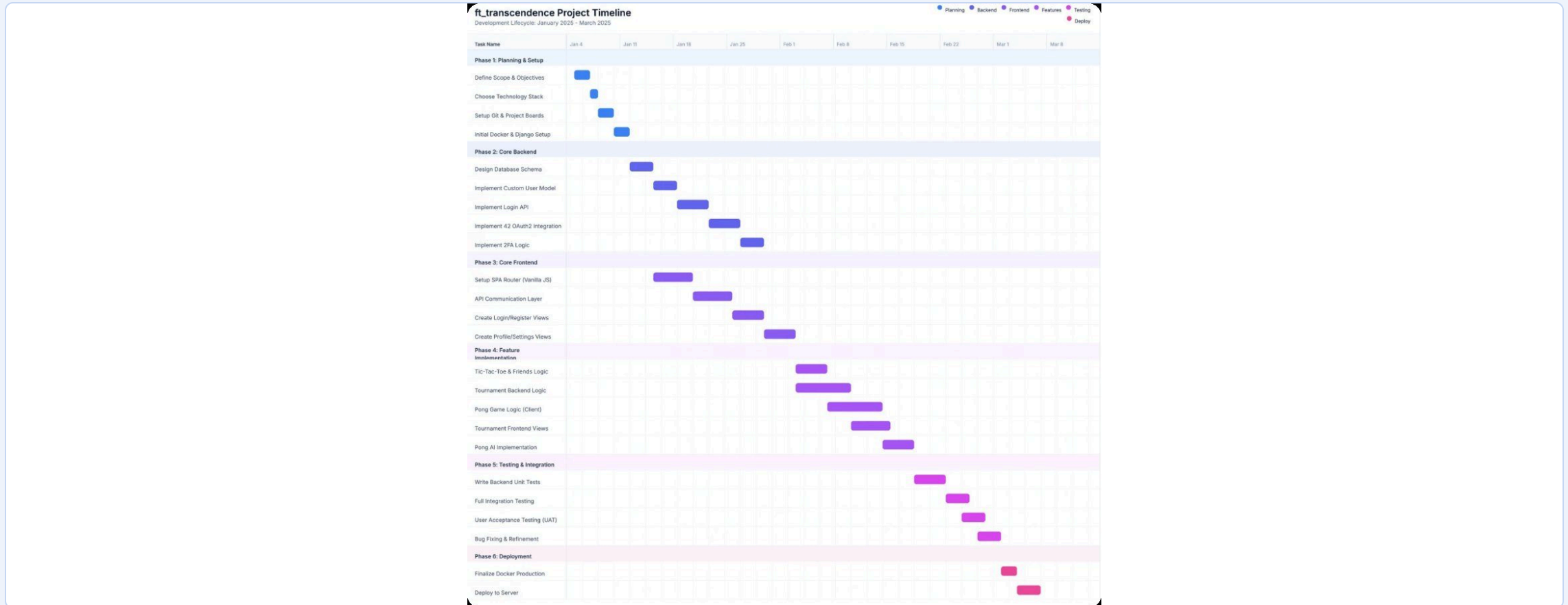


Figure 1: Project Gantt chart — planning & setup, core frontend, core backend, feature implementation, testing & integration, deployment.

03

Selected Modules & Design

Presenter: **Salim Hamzaoui**

Selected Modules

Category	Module	Type	How it is implemented
Web	Use a framework as backend	Major	Django 4.2 + Django REST Framework; apps userapp, gameapp, tournaments
Web	Use a front-end framework or toolkit	Minor	Bootstrap 4.5 + custom CSS; vanilla-JS SPA router
Web	Use a database for the backend	Minor	PostgreSQL 13 via the Django ORM and migrations
User Management	Standard user management, authentication, users across tournaments	Major	Register / login, profiles, avatars, friends, stats, match history; tournaments of nicknames
User Management	Implementing a remote authentication	Major	42 Intra OAuth 2.0 (signed state → authorize → callback → state check + server-side code exchange → JWT)
Gameplay & UX	Add another game with user history and matchmaking	Major	Tic-Tac-Toe (local, one device) with per-user match history; matchmaking = the tournament system (round-robin pairings, next-match announcement, tiebreakers)
AI-Algo	Introduce an AI opponent	Major	PongAI: looks once per second, predicts the intercept incl. wall bounces, presses simulated arrow keys at player speed — no A*
AI-Algo	User and game stats dashboards	Minor	Profile cards, win-rate chart, match history, tournament scoreboard, JSON export
Cybersecurity	GDPR compliance: anonymization, local data management, account deletion	Minor	Anonymization (42-safe), data export, edit data, account deletion, inactive-account cleanup
Cybersecurity	Two-Factor Authentication (2FA) and JWT	Major	Email one-time code on login; SimpleJWT access / refresh tokens
Graphics	Use of advanced 3D techniques	Major	Three.js: perspective camera, lights, Phong materials, textured spinning ball
Accessibility	Expanding browser compatibility	Minor	Standard web APIs only (ES modules, fetch, WebGL); tested in Chrome, Edge and Firefox
Accessibility	Server-Side Rendering (SSR) integration	Minor	Django pre-renders the requested page, title/meta and the profile data; the SPA hydrates it

7 Major + 6 Minor (× 0.5) = 10 major-equivalents — 7 are required for 100 %. Not selected: support on all devices (responsive layout and touch remain features), remote players, live chat, microservices, multiple languages.

System Architecture

The application is a monolith with a clear separation between frontend and backend, running in containers:

- The browser loads one server-rendered page and then runs the SPA.
- Gunicorn (3 workers) terminates HTTPS on port 443 and runs the Django app; WhiteNoise serves static files.
- Django exposes the REST API — accounts, presence, match history, tournaments (matchmaking) — and talks to PostgreSQL only through the ORM.
- External services: the 42 API for OAuth and Gmail SMTP for 2FA codes.
- Docker Compose defines the two services, the network and the database volume.

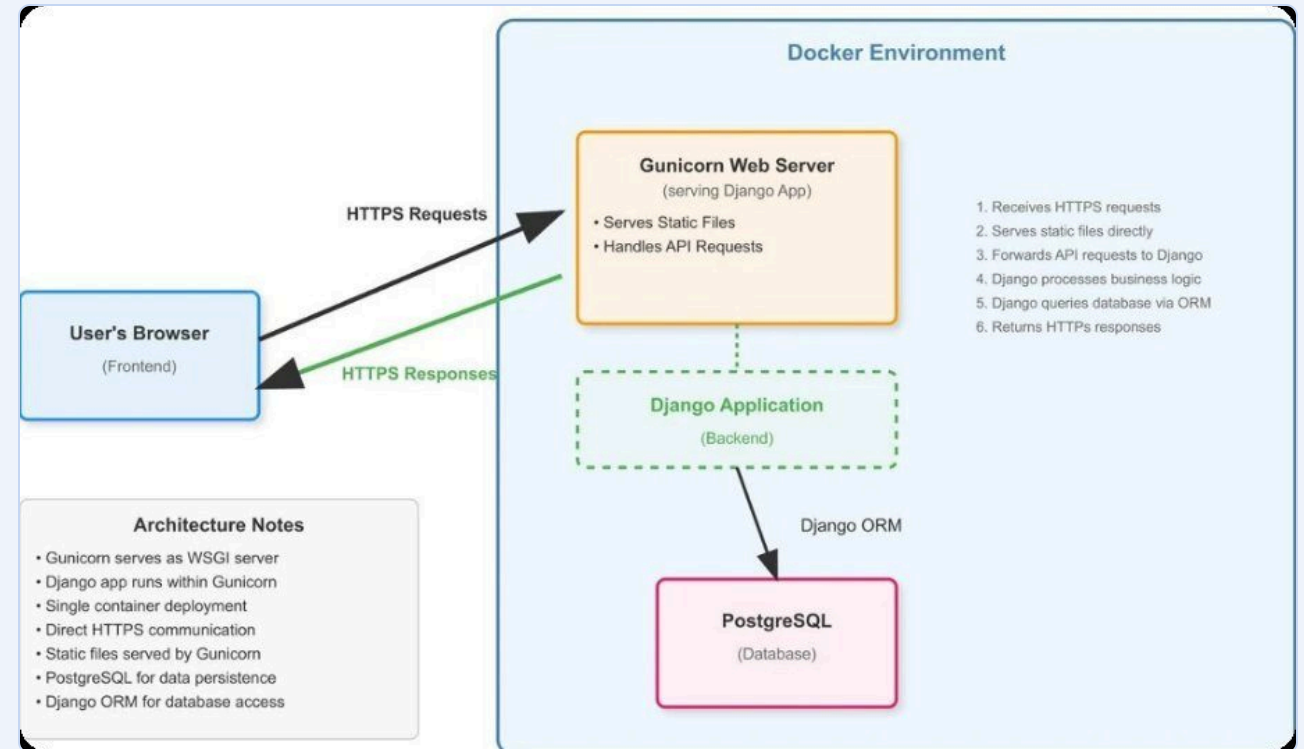
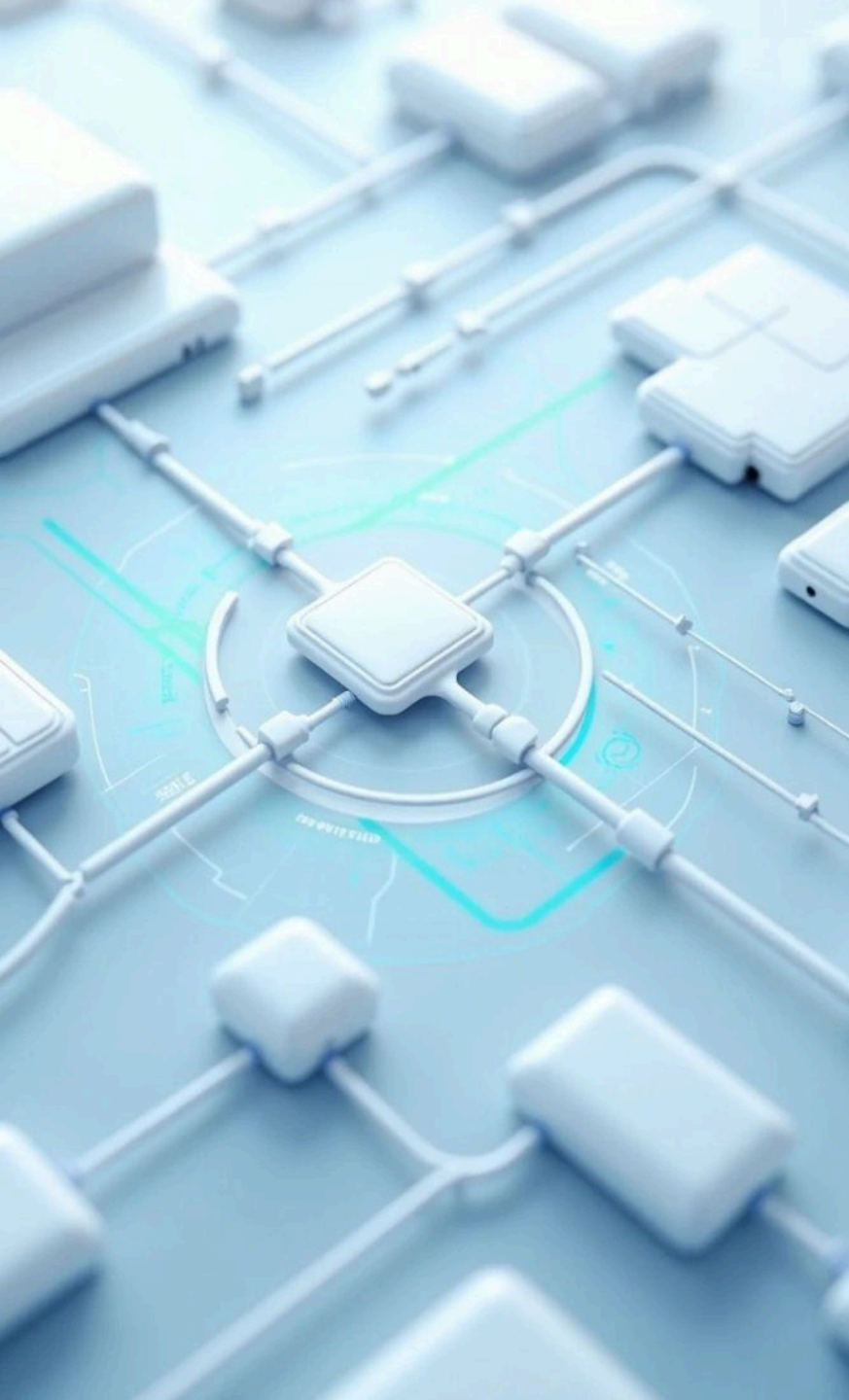


Figure 2: System architecture diagram



Database and API Design

The schema is defined with Django models, grouped into three apps; the frontend consumes a RESTful JSON API.

userapp

User (custom, email login, 2FA flag, avatar, friends M2M, last activity) and **MatchHistory** (game type, opponent, result, score, date).

gameapp

the **index** view that server-renders the requested page (SSR); both games run in the browser and post their results to MatchHistory. Legacy Game / Player / Score models.

tournaments

Tournament, **Player** (nickname per tournament) and **Match** (scores, winner, tiebreaker flag).

Endpoint	Purpose
POST /api/auth/register/ · login/ · logout/	Accounts and sessions (login starts 2FA)
POST /api/auth/verify-otp/ · token/refresh/	Check the emailed code, issue / refresh JWT
POST /api/auth/redirect_uri/ · get-token/	42 OAuth link and code exchange
GET/PUT /api/auth/profile/ · friends/... · users/	Profile, stats, avatar, unique display name, friends + online status
/api/auth/save-match/ · match-history/	Record and list games
/api/auth/export-data/ · anonymize-account/ · delete-account/	GDPR tools
POST /api/auth/heartbeat/	Presence: friends show online / offline (seen within 2 min)
/tournaments/api/tournaments/...	Create, add players, view, start / finish matches

UI / UX Design

The interface is built around clarity, simplicity and responsiveness: one consistent colour scheme and layout, immediate feedback on actions, and an app-like SPA experience.

SPA experience

No full-page reloads: the client-side router swaps views and keeps browser history working.

Responsive

Flexbox / grid layouts and media queries adapt the pages from desktop monitors to phones.

Feedback

Buttons react on hover, loading states are shown during API calls, results and errors are displayed clearly.

Wireframes & flowcharts

Screens and the gameplay / tournament flows were sketched before implementation.

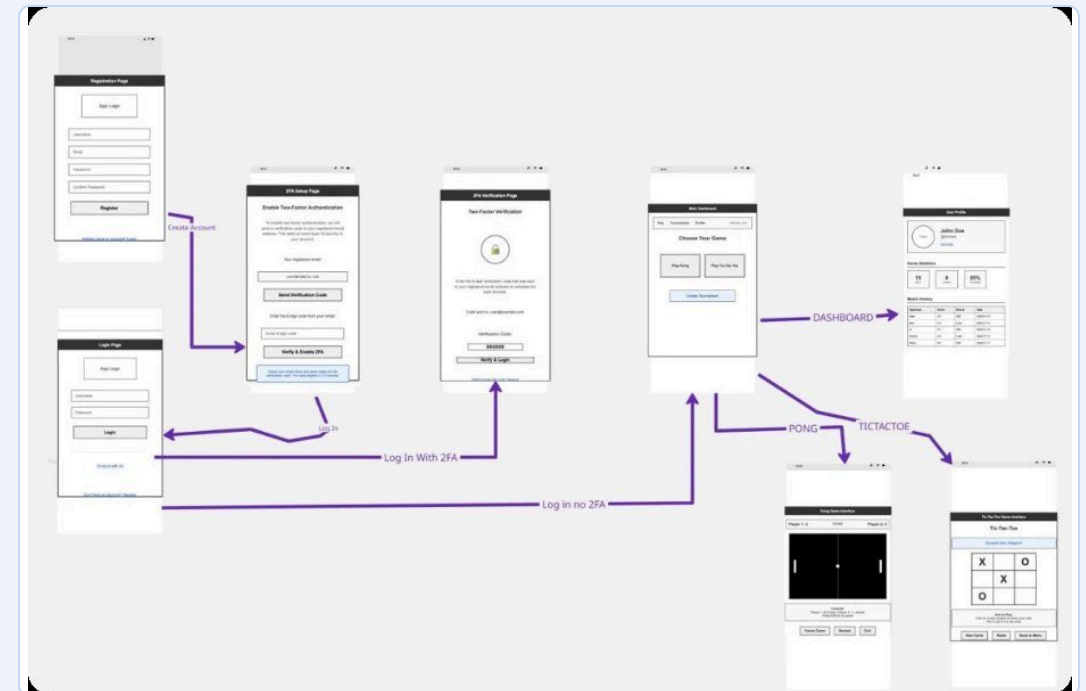


Figure 3: Wireframes of the key screens

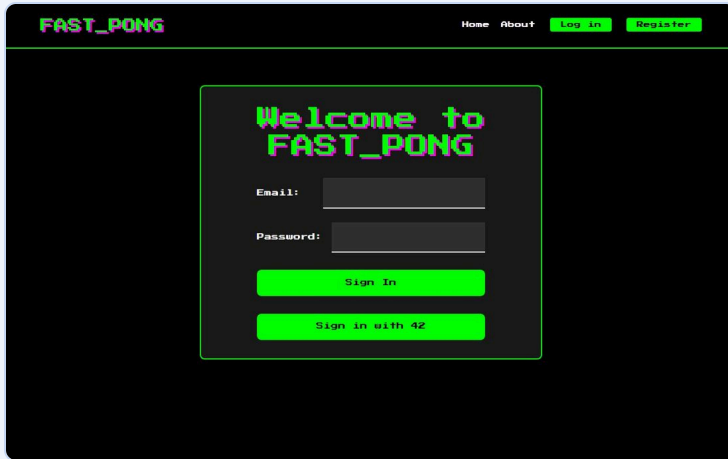
04

Authentication & Security

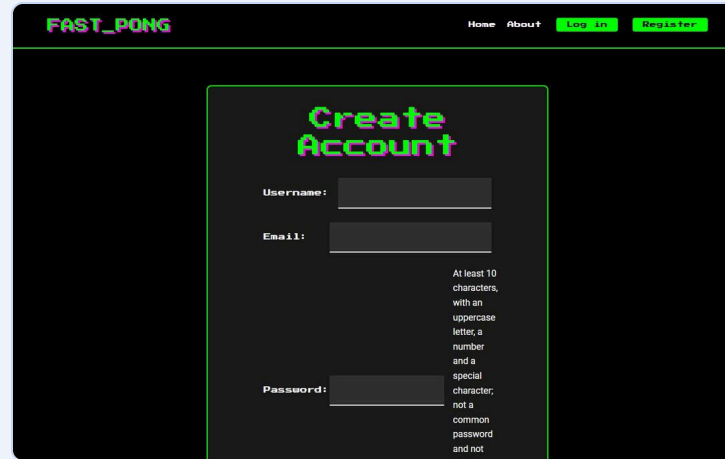
Presenter: **Nasser Alzaabi**

Features — Registration, Login and 2FA

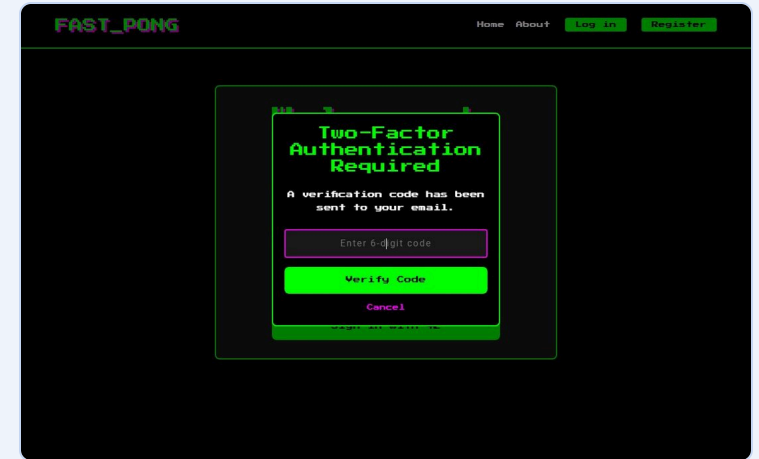
Email/password registration with a strong-password policy (every failing rule is reported separately), “Sign in with 42”, and an emailed one-time code when 2FA is enabled — password accounts switch it on or off in Settings; 42 accounts have no 2FA toggle.

The screenshot shows the login page of the FAST_PONG application. At the top, there is a navigation bar with the logo 'FAST_PONG' and links for 'Home', 'About', 'Log in', and 'Register'. The main content area features a central box with the heading 'Welcome to FAST_PONG'. Below the heading, there are two input fields: 'Email:' and 'Password:'. Under the password field, there are two buttons: 'Sign In' and 'Sign in with 42'.

Login page: email / password or “Sign in with 42”

The screenshot shows the registration page of the FAST_PONG application. It has the same navigation bar as the login page. The main content area has a central box with the heading 'Create Account'. Below the heading, there are three input fields: 'Username:', 'Email:', and 'Password:'. To the right of the password field, there is a list of password requirements: 'At least 10 characters, with an uppercase letter, a number and a special character; not a common password and not'.

Registration: rule-by-rule password feedback, optional two-factor authentication

The screenshot shows the 'Two-Factor Authentication Required' page of the FAST_PONG application. It has the same navigation bar. The main content area has a central box with the heading 'Two-Factor Authentication Required'. Below the heading, there is a message: 'A verification code has been sent to your email.' Below this message, there are three buttons: 'Enter 6-digit code', 'Verify Code', and 'Cancel'.

Second factor: 6-digit code sent by email, valid 10 minutes, single use

How a Login Works — 42 OAuth, 2FA and JWT

Three flows share one outcome: a logged-in user with a session and a JWT pair.

1 Password + 2FA

1. POST /login/ with email and password → Django `authenticate()` (PBKDF2 hash check).
2. If 2FA is on: a 6-digit code is stored in a **shared database cache** for 10 min and emailed from a background thread; the response is *requires_2fa*.
3. POST /verify-otp/ → code compared and deleted (single use) → session + JWT pair returned.

2 Remote authentication (42)

1. “Sign in with 42” → the backend builds the authorize URL (client id, redirect URI, *response_type=code*) plus a signed, time-limited *state* kept in the session.
2. The student consents on the 42 Intra; 42 redirects to `/oauth/callback?code=...&state=...` in our SPA.
3. The SPA posts code + state; the **server** checks the state (single use, 10 min — CSRF protection), exchanges the code with the client secret (never in the browser), reads `/v2/me`, creates or links the user by email, logs in, returns JWTs.

3 JSON Web Tokens

1. SimpleJWT: access token 60 min, refresh token 7 days, signed HS256 with the Django secret; claims *user_id*, *exp*, *iat*, *jti*.
2. The SPA sends Authorization: Bearer on every API call (games, profile, friends, GDPR).
3. `authFetch` refreshes the token one minute before expiry and retries once after a 401, so a session survives the 60-minute limit.

Both paths end in the same state: a Django session cookie (HttpOnly) plus a JWT pair — 42 users and password users are indistinguishable afterwards.



Cybersecurity Features

Security was a fundamental requirement, addressed in layers from the database to the browser.

Password storage

Django hashes and salts every password (PBKDF2); validators enforce length, upper-case, digit and symbol and report each failing rule.

2FA + JWT

Optional emailed one-time code as a second factor; SimpleJWT access (60 min) and refresh (7 days) tokens.

42 OAuth

Students log in through the 42 Intra; the server verifies the signed *state*, exchanges the code and never sees a password.

SQL injection

All database access goes through the ORM, which parameterises every query.

CSRF & XSS

Django's CSRF token is required on every state-changing request; user data (names, nicknames) is inserted with `textContent`, never as HTML.

Transport & access control

HTTPS everywhere — Gunicorn terminates TLS on port 443; every game, tournament and profile API requires login; secrets live in `.env`.

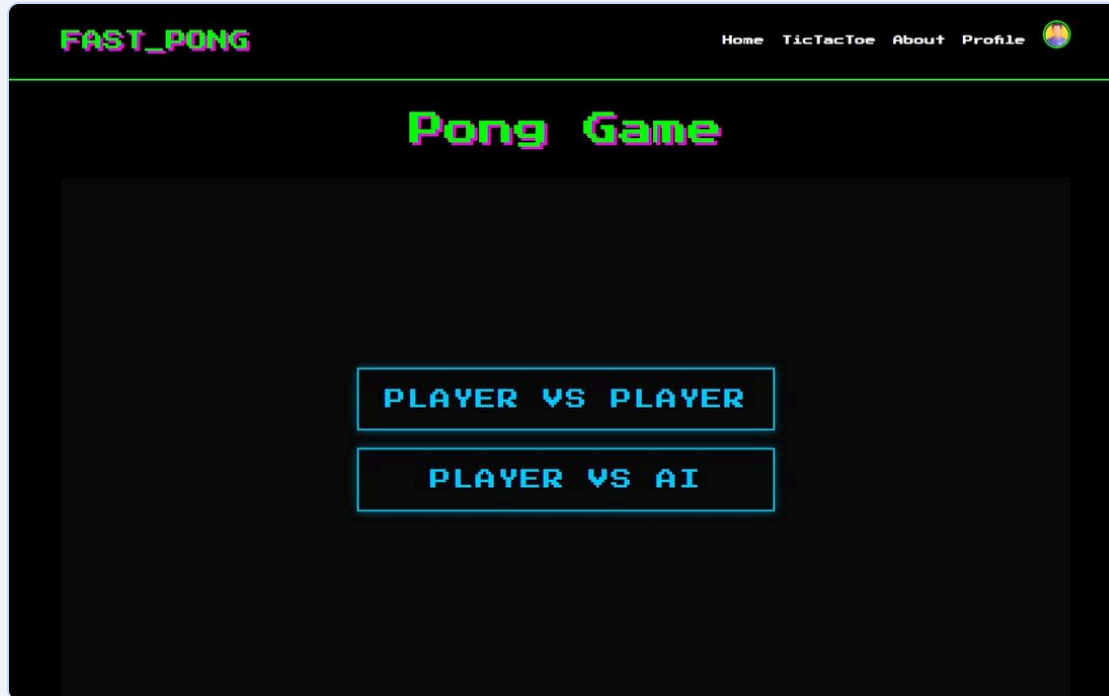
05

Games & Graphics

Presenter: **Salim Hamzaoui**

Features — 3D Pong and the AI Opponent

Three.js scene with a perspective camera, lit table and a spinning textured ball. In “Player vs AI” the PongAI looks at the ball once per second, predicts where it will cross the paddle line and presses simulated arrow keys — first to 3 points wins.



Mode selection: Player vs Player (one keyboard or touch) or Player vs AI



Game in progress against the AI

Inside the Graphics and the AI Opponent

3D scene (Three.js / WebGL)

PerspectiveCamera (75°) looking down at a glossy table with neon edge lines; ambient + spot lighting; Phong materials with specular highlights; emissive cyan paddles; a magenta ball whose stripes are painted on an HTML canvas and used as a texture; antialiased renderer; a DOM HUD over the canvas.

Physics

The bounce angle depends on where the ball hits the paddle (up to $\pm 45^\circ$); the ball speeds up 5 % per hit; spin bends the trajectory and rotates the mesh; walls clamp and reflect; the canvas keeps a 4:3 ratio on resize; keyboard and touch input.

AI — look once per second

Every 1000 ms the AI takes one snapshot of the ball position and velocity (the subject's "refresh once per second" rule). Between snapshots it acts only on its last decision.

AI — predict, with errors

It extrapolates where the ball will cross its paddle line, folding the path at the walls so bounces are anticipated; then adds a small prediction error and, 10 % of the time, a big "mistake".

AI — simulated keyboard

The decision becomes a held ArrowUp / ArrowDown key fed into the same InputHandler as a human, so the AI paddle moves at exactly human speed. Every 5 s the score adjusts its judgement: it eases off when 2 points ahead, tries harder when behind. No A* — Pong has no graph to search.

Features — Tic-Tac-Toe: the Second Game, History and Matchmaking

The “Add another game” module: a new game distinct from Pong, with user history tracking and a matchmaking system.



The 3 × 3 board: two players on one device, X and O alternate, “Reset game” starts over

A game distinct from Pong

Turn-based Tic-Tac-Toe played locally on one device — like Pong, no online play by design. Win lines and draws are detected in the browser; moves on an occupied cell or after the end are ignored.

User history tracking

Every finished game is posted to `/api/auth/save-match/` with the JWT and stored as a MatchHistory row (TICTACTOE, WIN / LOSS / DRAW, date), so it appears in the profile’s recent matches, the win-rate chart and the JSON export.

Matchmaking

Matchmaking is the tournament system: a tournament of 3–8 players is scheduled as a round-robin, the next pairing is announced (“Next match: A vs B”) and tied leaders are matched again in tiebreaker rounds until one winner remains.

Not online

Both games run in the browser on one device; there is no online or remote play in the project — the “remote players” module is not selected.

06

Tournaments

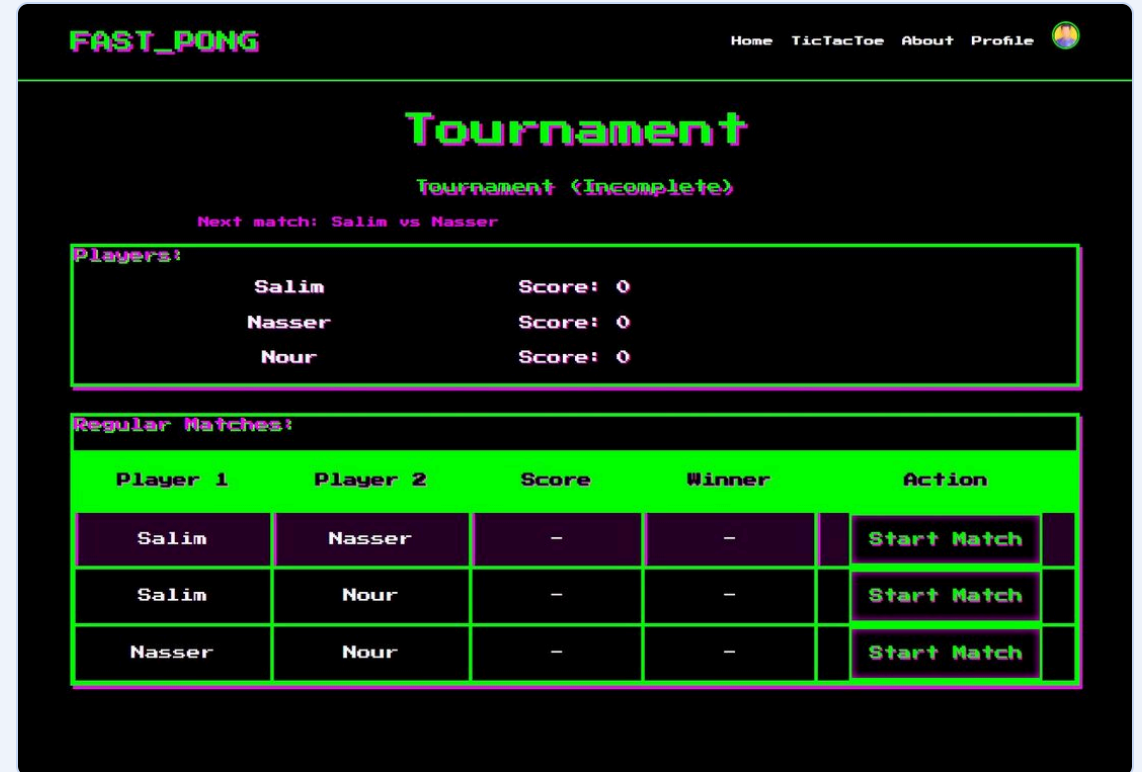
Presenter: **Nour Murat**

Features — Tournaments

A logged-in user creates a tournament of 3–8 nicknames; every pair plays one Pong match, scores update the table, and tied leaders get automatic tiebreaker matches until one winner remains.



Create a tournament and register the nicknames (unique inside the tournament)



Player 1	Player 2	Score	Winner	Action
Salim	Nasser	–	–	Start Match
Salim	Nour	–	–	Start Match
Nasser	Nour	–	–	Start Match

Round-robin schedule with “Start Match” per pairing, “Next match” announcement, live scores and the winner

Tournament Flow

From creation to the final winner, the tournament system runs six steps:

1 · Create

A logged-in user names the tournament and chooses 3–8 players.

2 · Register

Each player types a nickname; duplicates and empty names are rejected.

3 · Schedule

The server generates the round-robin: every pair meets exactly once.

4 · Play

“Start Match” opens 3D Pong with both nicknames on screen (two players, one keyboard).

5 · Score

The result is posted to the tournament API; the table shows points per player and announces the next match.

6 · Winner

If leaders are tied, extra tiebreaker matches are created until one player wins.

Users across tournaments

Nicknames belong to one tournament, so a person can join many tournaments with different aliases; the account that created the tournament must be logged in.

Kept safe

All tournament endpoints require login and the CSRF token; nicknames are shown as text (no HTML injection); scores must be non-negative integers and a match cannot end in a tie.

Stays after refresh

The current tournament id is remembered in the browser, so reloading the page returns to the same tournament.

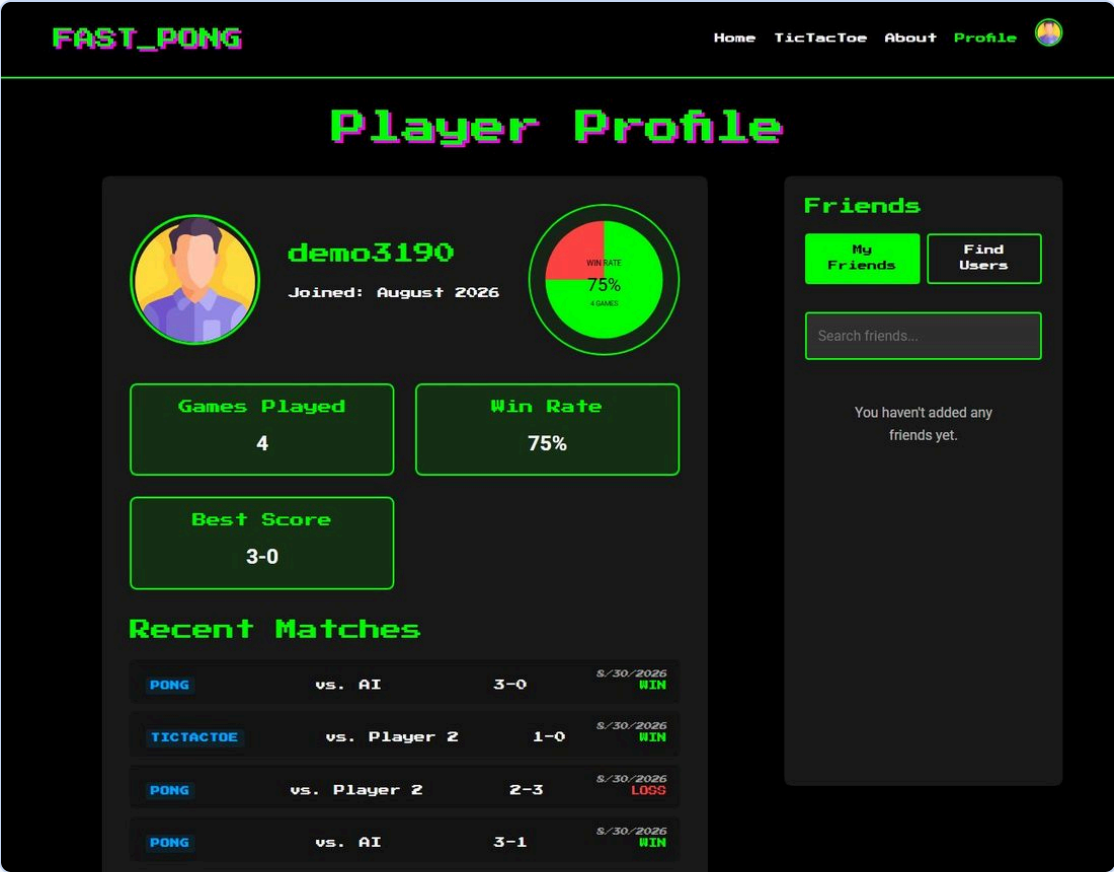
07

Profiles, Statistics & Friends

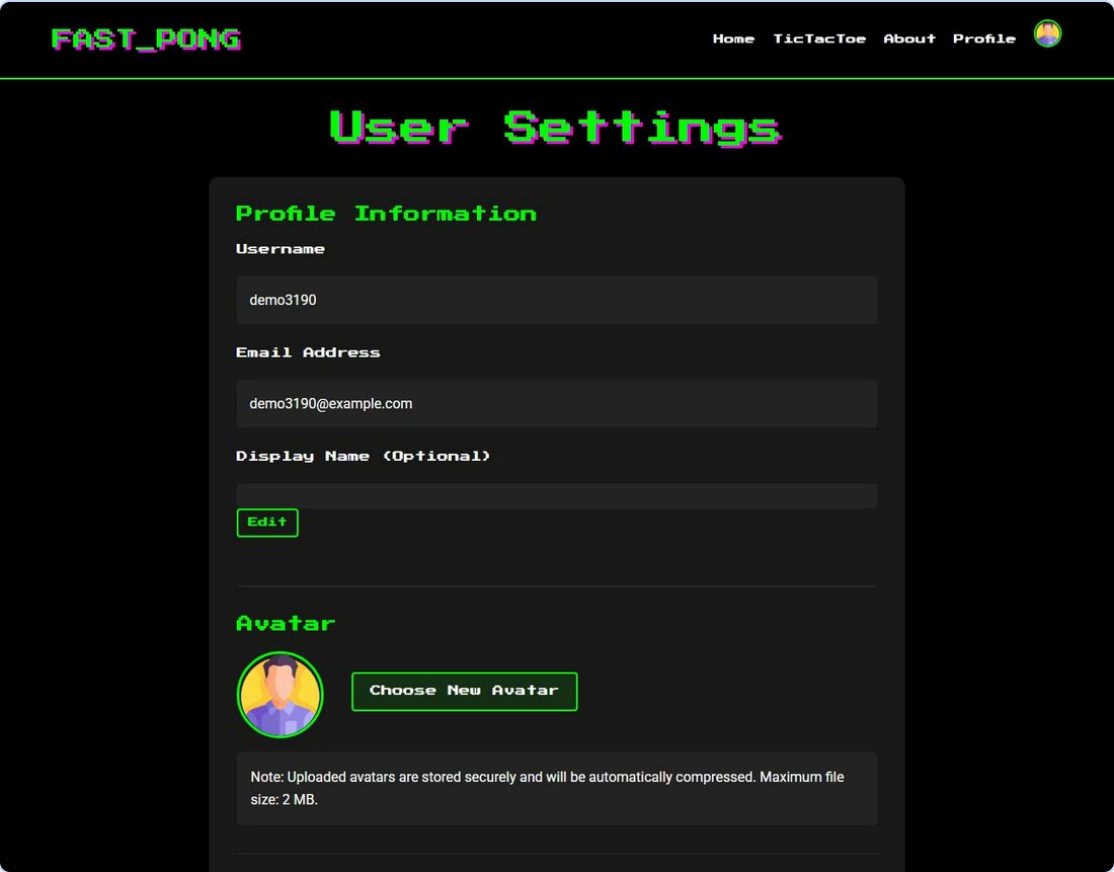
Presenter: **Alisher Abdullaev**

Features — Player Profile and Stats Dashboard

Every finished game — Pong, Pong vs AI, Tic-Tac-Toe — is recorded; the profile turns the history into a dashboard and the settings page holds the account and GDPR tools.



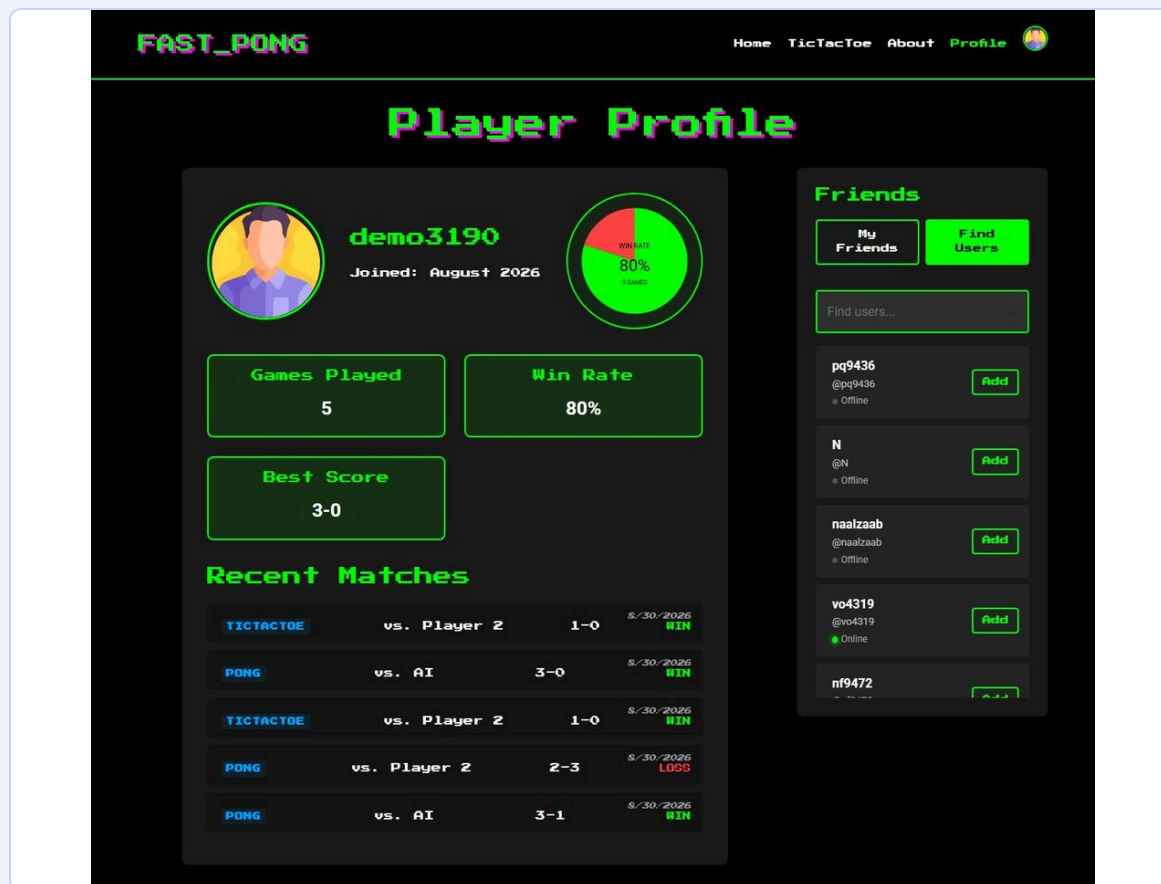
Games played, win-rate chart, best score, recent matches with game type and date, friends panel with online / offline dots



Display name, e-mail, avatar, 2FA switch (password accounts), Download my data, Anonymize, Delete account

Features — Friends, Match History and User Data

The user-management and stats-dashboard modules, seen from the user's side.



“Find Users” lists every active player with an Add / Remove friend button

User information

Custom Django user: e-mail is the login, unique username, optional display name (unique, case-insensitive), avatar (default picture if none, max 2 MB, image type checked), 2FA flag, 42 link.

Friends

Add or remove friends from the Find Users list; the friends panel shows names, avatars and an online / offline dot — the SPA sends a heartbeat every minute, a user counts as online when seen in the last 2 minutes, and logging out sets offline at once. Presence only: there is no online play.

Match history

One row per game: type (PONG / TICTACTOE), opponent, score, result, date. Last five on the profile, full list in the JSON export.

Statistics

Games played, win rate (hand-drawn SVG pie chart), best score = the win with the largest margin. Tournament games are kept separate because their players are nicknames, not accounts.

08

GDPR & Accessibility

Presenter: **Nasser Alzaabi**



GDPR Compliance

Users own their data: they can see it, take it with them, anonymize it or erase the account — and inactive accounts are cleaned up automatically.

Anonymization

“Anonymize My Account” replaces username, e-mail, avatar and 42 link with anonymous values, clears friends and disables login; the non-personal game statistics are kept. Works for 42 accounts too.

Local data management

“Download my data” exports profile, statistics and full match history as JSON; display name, e-mail and avatar can be edited in Settings.

Account deletion

Hard-deletes the account and everything attached to it (match history, friend links, tokens) in one click, after confirmation.

Retention

A management command warns after 5 months of inactivity and deletes after 6 (last activity tracked by middleware).

The privacy policy on the About page describes the data collected, its use, retention and the user’s rights.

Accessibility — Browser Compatibility and Server-Side Rendering

Expanding browser compatibility

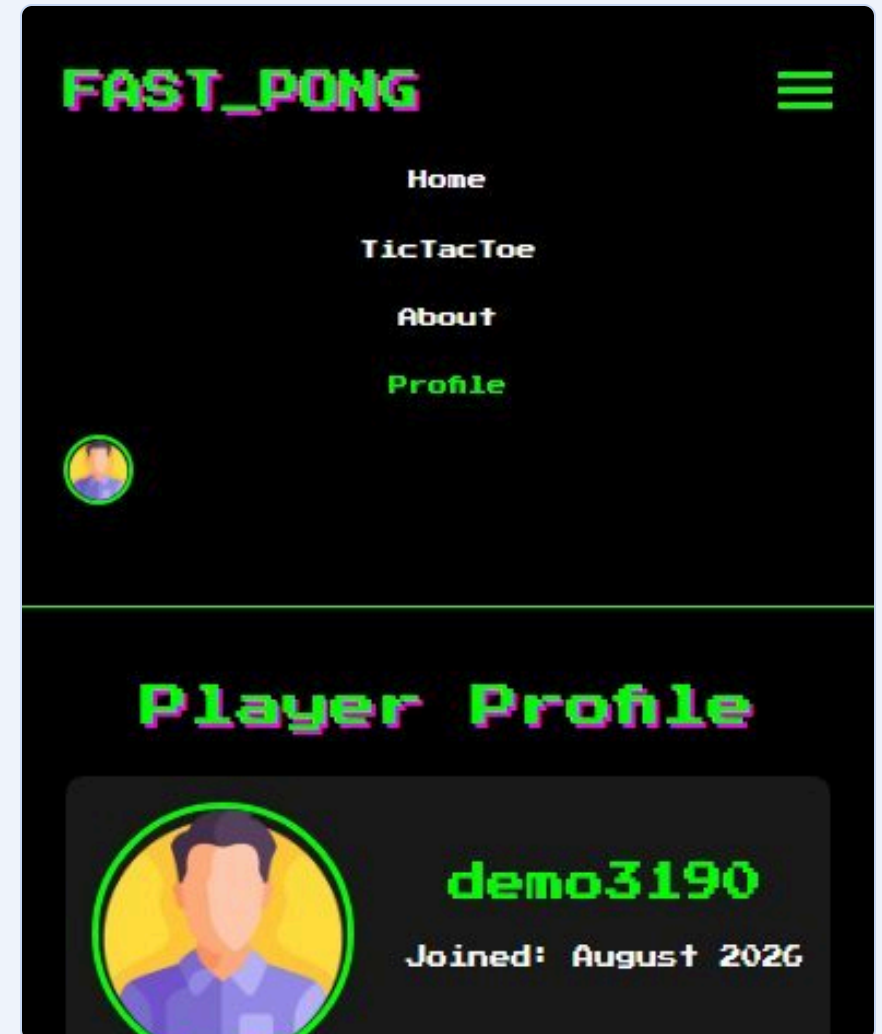
Primary browser: Chrome (and Edge). Additional browser: Firefox. The app uses only standard web APIs — ES modules, fetch, localStorage, History API, CSS Grid / Flexbox, WebGL 1 — with no vendor prefixes or polyfills. Browser-specific fixes: ISO-8601 dates so Firefox parses them, pointer events instead of touch/mouse events, font fallbacks. Tested manually in both browsers and with an automated headless-Chrome walkthrough of every page.

Server-Side Rendering

Every URL is answered by Django's index view, which renders the *requested* page as complete HTML: the right section is already active, the navigation reflects the login state, the page title and meta description are set, and for a logged-in user the profile — username, statistics, recent matches — is rendered into the HTML before any JavaScript runs. The SPA then hydrates and takes over routing. View-source shows real content, not an empty `<div>`, which helps first paint and SEO.

Responsive layout (feature)

Media queries at 1100 / 920 / 768 / 480 px, hamburger navigation, stacked cards, a fluid game canvas and touch controls for Pong — kept as a feature although “support on all devices” is not a claimed module.



Profile on a 390 px phone

09

Team & Conclusion

Presenter: **Alisher Abdullaev**



Contribution of Each Member

Each member owned a group of modules end-to-end — backend, frontend and tests — and reviewed the others' pull requests.

Salim Hamzaoui

Graphics — 3D Pong with Three.js and its physics;
Gameplay — the second game, Tic-Tac-Toe, with match history and tournament matchmaking; **Cybersecurity & GDPR** — anonymization, data export, account deletion, retention cleanup, privacy policy.

Nasser Alzaabi

Authentication — registration, login, sessions; **Remote authentication** — the 42 OAuth 2.0 flow; **2FA and JWT** — emailed one-time codes, SimpleJWT access / refresh tokens and their automatic renewal in the SPA.

Alisher Abdullaev

User history and statistics — match history, stats dashboard, win-rate chart; **user information** — username, avatar, display name, e-mail, friends;
Database module — PostgreSQL, models and migrations.

Nour Murat

Mandatory part — the tournament system with round-robin scheduling and tiebreakers; **Front end** — the SPA and the Bootstrap toolkit module; **Backend framework module** — the Django project structure and REST API.



Conclusion

Ft_transcendence delivered a feature-rich, secure and engaging web gaming platform that satisfies all the selected modules — 7 Major and 6 Minor, 10 major-equivalents against the 7 required.

Using Django, a vanilla-JavaScript SPA with Bootstrap, Three.js and PostgreSQL inside Docker, the team gained hands-on experience in full-stack development, game logic, deployment and application security. The Agile, feature-branch workflow kept four developers productive and the codebase reviewable.

The modular structure is a solid base for the next evolution — remote play, chat and a richer AI — while the pre-evaluation audit leaves the project with a green test suite and documented, root-caused fixes.

The background is a light blue gradient with various abstract shapes. A large, translucent blue ring with a white center is positioned in the middle. Inside and around this ring are numerous smaller, translucent blue and purple bubbles of different sizes. Some bubbles have darker blue or purple centers, giving them a 3D effect. The overall aesthetic is clean, modern, and aquatic.

Thank You